



THE COMPLETE PRACTICE HANDBOOK

300 SQL Problems & Solutions

A Scenario-Based Guide for Beginners to Advanced
Programmers



300

PROBLEMS

15

SECTIONS

8

REF. TABLES

*"From SELECT * to window functions and CTEs — every query explained
with a real schema"*

SQL PRACTICE SERIES

Employees · Orders · Products Schema

@KartikMungole

Table of Contents

15 sections, progressing from basic *SELECT*s to advanced real-world dashboards

Every problem follows the same format: a **real-world scenario**, a **difficulty tag**, and a **ready-to-run SQL solution**. All problems use the shared schema shown on the next page, so you always know exactly which tables and columns you're working with.

01 SELECT Basics	Q1-Q20
Learn to retrieve data from tables using <i>SELECT</i> — the most fundamental SQL command.	
02 WHERE & Filtering	Q21-Q45
Filter rows using <i>WHERE</i> with comparison, logical, and pattern-matching operators.	
03 ORDER BY & LIMIT	Q46-Q60
Sort results and limit output — essential for reports and pagination.	
04 Aggregate Functions	Q61-Q85
Summarize data using <i>COUNT</i> , <i>SUM</i> , <i>AVG</i> , <i>MIN</i> , <i>MAX</i> and more.	
05 GROUP BY & HAVING	Q86-Q110
Group rows and filter groups — essential for reporting and analytics.	
06 JOINS	Q111-Q145
Combine data from multiple tables — the heart of relational databases.	
07 Subqueries	Q146-Q170
Queries within queries — for complex filtering, derived tables, and computed values.	
08 String Functions	Q171-Q190
Manipulate and analyze text data within SQL.	
09 Date & Time Functions	Q191-Q210
Work with dates and timestamps for time-based analysis and reporting.	

Table of Contents

10 CASE & Conditional Logic	Q211–Q225
Add if-then-else logic directly in your SQL queries.	
11 Window Functions	Q226–Q250
Perform calculations across rows without collapsing groups — powerful analytics tool.	
12 INSERT, UPDATE, DELETE	Q251–Q265
Modify data in tables — the write side of SQL.	
13 Table Design & DDL	Q266–Q280
Create, modify, and manage table structures.	
14 Views, Indexes & Optimization	Q281–Q290
Performance tuning and reusable query definitions.	
15 Advanced & Real-World Problems	Q291–Q300
Complex real-world scenarios combining multiple SQL concepts.	

Reference Database Schema

The 8 tables used throughout this handbook, with sample data

i Every query in this guide runs against this same small e-commerce/HR-style database. When a problem says "find employees in IT," it means the `employees` table shown below — not an abstract table. Sample rows are illustrative; your actual result sets will differ.

employees

PK: employee_id

Core staff table used in almost every problem — salaries, departments, managers.

department_id → departments.department_id

manager_id → employees.employee_id (self-ref)

employee_id	name	email	department	department_id
1	Aditi Sharma	aditi.sharma@techcorp.com	IT	2
2	Rahul Verma	rahul.verma@techcorp.com	IT	2
3	Neha Gupta	neha.gupta@techcorp.com	HR	1
4	Karan Singh	karan.singh@techcorp.com	Sales	3
5	Priya Nair	priya.nair@techcorp.com	Sales	3

departments

PK: department_id

Lookup table of company departments, referenced by employees.department_id.

department_id	department_name	location
1	HR	Mumbai
2	IT	Bengaluru
3	Sales	Delhi
4	Finance	Pune
5	Marketing	Chennai

Reference Database Schema

customers

PK: customer_id

People or companies who place orders — the buyer side of the sales schema.

customer_id	customer_name	email	city	country
1	Rohan Mehta	rohan.mehta@mail.com	Mumbai	India
2	Sara Khan	sara.khan@mail.com	Delhi	India
3	Global Traders Ltd	contact@globaltraders.com	New York	USA
4	Ananya Iyer	ananya.iyer@mail.com	Chennai	India
5	Blue Ocean Retail	info@blueocean.com	London	UK

orders

PK: order_id

One row per order placed by a customer; amount is the order total.

customer_id → customers.customer_id

order_id	customer_id	order_date	amount	status
101	1	2024-01-12	2500.00	Delivered
102	2	2024-02-03	1200.50	Pending
103	1	2024-02-18	4300.00	Delivered
104	3	2024-03-05	980.00	Cancelled
105	4	2024-03-21	1560.75	Shipped

Reference Database Schema

order_items

PK: order_item_id

Line items within an order — the many-to-many bridge between orders and products.

order_id → orders.order_id

product_id → products.product_id

order_item_id	order_id	product_id	quantity	price
1	101	1	2	450.00
2	101	3	1	1600.00
3	102	2	3	400.00
4	103	1	4	450.00
5	104	4	1	980.00

products

PK: product_id

Catalogue of items that can be sold, linked to a category and to order_items.

category_id → categories.category_id

product_id	product_name	category_id	price	stock_quantity
1	Wireless Mouse	1	450.00	120
2	USB-C Cable	1	400.00	300
3	Office Chair	2	1600.00	45
4	Standing Desk	2	980.00	20
5	Notebook Pack	3	150.00	500

Reference Database Schema

categories

PK: category_id

Groups products into categories for reporting (e.g. Q122 total sales per category).

category_id	category_name
1	Electronics
2	Furniture
3	Stationery

suppliers

PK: supplier_id

Vendors who supply products — used in a handful of advanced/DDL problems.

supplier_id	supplier_name	city	country
1	TechSource Pvt Ltd	Shenzhen	China
2	OfficeWorld Inc	Chicago	USA
3	PaperPlus Co	Mumbai	India

SELECT Basics

Learn to retrieve data from tables using *SELECT* — the most fundamental SQL command.

Q1. Select All Columns

Easy

Write a query to fetch all records from the **employees** table.

```
SELECT *  
FROM employees;
```

Q2. Select Specific Columns

Easy

Fetch only **name** and **salary** from the employees table.

```
SELECT name, salary  
FROM employees;
```

Q3. Column Alias

Easy

Display employee name as "Employee Name" and salary as "Monthly Pay".

```
SELECT name AS "Employee Name",  
       salary AS "Monthly Pay"  
FROM employees;
```

Q4. DISTINCT Values

Easy

Get all unique department names from the employees table.

```
SELECT DISTINCT department  
FROM employees;
```

SELECT Basics

Q5. Arithmetic in SELECT

Easy

Show each employee's annual salary (salary × 12).

```
SELECT name,  
       salary,  
       salary * 12 AS annual_salary  
FROM employees;
```

Q6. Concatenate Columns

Easy

Combine first_name and last_name into a single "full_name" column.

```
SELECT first_name || ' ' || last_name AS full_name  
FROM employees;  
  
-- MySQL alternative:  
SELECT CONCAT(first_name, ' ', last_name) AS full_name  
FROM employees;
```

Q7. Select with Constants

Easy

Show each employee's name with a constant column "Company" = 'TechCorp'.

```
SELECT name,  
       'TechCorp' AS company  
FROM employees;
```

Q8. Count Total Rows

Easy

How many employees are in the table?

```
SELECT COUNT(*) AS total_employees  
FROM employees;
```

SELECT Basics

Q9. Select Without Table

Easy

Calculate 15% tax on a salary of 50000 without using any table.

```
SELECT 50000 AS salary,  
       50000 * 0.15 AS tax,  
       50000 * 0.85 AS after_tax;
```

Q10. Select Current Date

Easy

Get the current date and time from the database.

```
-- PostgreSQL / MySQL  
SELECT CURRENT_DATE AS today,  
       CURRENT_TIMESTAMP AS now;  
  
-- SQL Server  
SELECT GETDATE() AS now;
```

Q11. COALESCE for NULL Handling

Easy

Show employee name and bonus. If bonus is NULL, show 0.

```
SELECT name,  
       COALESCE(bonus, 0) AS bonus  
FROM employees;
```

Q12. NULLIF Function

Easy

Return NULL if salary equals 0 (to avoid division by zero later).

```
SELECT name,  
       NULLIF(salary, 0) AS salary  
FROM employees;
```

SELECT Basics

Q13. Select Top N Rows

Easy

Get the first 5 employees from the table.

```
-- PostgreSQL / MySQL
SELECT * FROM employees LIMIT 5;

-- SQL Server
SELECT TOP 5 * FROM employees;
```

Q14. Percentage Calculation

Easy

Show each product's price and its 18% GST amount.

```
SELECT product_name,
       price,
       ROUND(price * 0.18, 2) AS gst,
       ROUND(price * 1.18, 2) AS price_with_gst
FROM products;
```

Q15. Multiple DISTINCT Columns

Easy

Get all unique combinations of department and city.

```
SELECT DISTINCT department, city
FROM employees;
```

Q16. Boolean Expression in SELECT

Easy

Show each employee and whether their salary is above 50000.

```
SELECT name,
       salary,
       (salary > 50000) AS is_high_earner
FROM employees;
```

SELECT Basics

Q17. Type Casting

Easy

Convert salary (integer) to text for display purposes.

```
-- PostgreSQL
SELECT name, salary::TEXT AS salary_text
FROM employees;

-- MySQL
SELECT name, CAST(salary AS CHAR) AS salary_text
FROM employees;
```

Q18. Select with Math Functions

Easy

Show the absolute value, ceiling, and floor of a discount column.

```
SELECT product_name,
       discount,
       ABS(discount) AS abs_discount,
       CEIL(discount) AS ceiling_val,
       FLOOR(discount) AS floor_val
FROM products;
```

Q19. Power and Square Root

Easy

Calculate the square and square root of each product's price.

```
SELECT product_name,
       price,
       POWER(price, 2) AS price_squared,
       ROUND(SQRT(price), 2) AS price_sqrt
FROM products;
```

SELECT Basics

Q20. Modulo Operator

Easy

Find employees whose ID is an even number.

```
SELECT employee_id, name
FROM employees
WHERE employee_id % 2 = 0;
```

WHERE & Filtering

Filter rows using *WHERE* with comparison, logical, and pattern-matching operators.

Q21. Simple WHERE

Easy

Find employees in the 'Sales' department.

```
SELECT * FROM employees
WHERE department = 'Sales';
```

Q22. Greater Than

Easy

Find employees earning more than 60000.

```
SELECT name, salary
FROM employees
WHERE salary > 60000;
```

Q23. AND Operator

Easy

Find employees in 'IT' department earning above 50000.

```
SELECT name, department, salary
FROM employees
WHERE department = 'IT'
AND salary > 50000;
```

Q24. OR Operator

Easy

Find employees who are in 'Sales' OR 'Marketing'.

```
SELECT name, department
FROM employees
WHERE department = 'Sales'
OR department = 'Marketing';
```

WHERE & Filtering

Q25. NOT Operator

Easy

Find all employees NOT in the 'HR' department.

```
SELECT name, department
FROM employees
WHERE NOT department = 'HR';

-- Alternative:
WHERE department ≠ 'HR';
-- or
WHERE department <> 'HR';
```

Q26. BETWEEN

Easy

Find employees with salary between 30000 and 60000 (inclusive).

```
SELECT name, salary
FROM employees
WHERE salary BETWEEN 30000 AND 60000;
```

Q27. IN Operator

Easy

Find employees in departments: IT, Sales, or Finance.

```
SELECT name, department
FROM employees
WHERE department IN ('IT', 'Sales', 'Finance');
```

Q28. NOT IN

Easy

Find employees NOT in IT or HR.

```
SELECT name, department
FROM employees
WHERE department NOT IN ('IT', 'HR');
```

WHERE & Filtering

Q29. LIKE — Starts With

Easy

Find employees whose name starts with 'A'.

```
SELECT name FROM employees
WHERE name LIKE 'A%';
```

Q30. LIKE — Ends With

Easy

Find employees whose name ends with 'kumar'.

```
SELECT name FROM employees
WHERE name LIKE '%kumar';
```

Q31. LIKE — Contains

Easy

Find employees whose name contains 'raj'.

```
SELECT name FROM employees
WHERE name LIKE '%raj%';
```

Q32. LIKE — Single Character Wildcard

Easy

Find employees whose name is 4 letters and starts with 'R'.

```
SELECT name FROM employees
WHERE name LIKE 'R___';
```

WHERE & Filtering

Q33. ILIKE — Case Insensitive (PostgreSQL)

Easy

Find employees whose name contains 'john' regardless of case.

```
-- PostgreSQL
SELECT name FROM employees
WHERE name ILIKE '%john%';

-- MySQL (LIKE is case-insensitive by default)
SELECT name FROM employees
WHERE name LIKE '%john%';
```

Q34. IS NULL

Easy

Find employees who don't have a manager assigned.

```
SELECT name FROM employees
WHERE manager_id IS NULL;
```

Q35. IS NOT NULL

Easy

Find employees who have an email address.

```
SELECT name, email FROM employees
WHERE email IS NOT NULL;
```

Q36. Multiple Conditions

Medium

Find employees in IT dept, salary > 40000, and joined after 2020.

```
SELECT name, department, salary, join_date
FROM employees
WHERE department = 'IT'
  AND salary > 40000
  AND join_date > '2020-01-01';
```

WHERE & Filtering

Q37. Parentheses for Logic

Medium

Find employees who are in (Sales OR Marketing) AND earn above 50000.

```
SELECT name, department, salary
FROM employees
WHERE (department = 'Sales' OR department = 'Marketing')
AND salary > 50000;
```

Q38. NOT LIKE

Easy

Find employees whose email does NOT end with '@gmail.com'.

```
SELECT name, email FROM employees
WHERE email NOT LIKE '%@gmail.com';
```

Q39. NOT BETWEEN

Easy

Find employees whose salary is NOT between 30000 and 50000.

```
SELECT name, salary FROM employees
WHERE salary NOT BETWEEN 30000 AND 50000;
```

Q40. Filter on Date

Easy

Find orders placed on '2024-01-15'.

```
SELECT order_id, customer_name, order_date
FROM orders
WHERE order_date = '2024-01-15';
```

WHERE & Filtering

Q41. Filter Date Range

Easy

Find orders placed in January 2024.

```
SELECT order_id, order_date, amount
FROM orders
WHERE order_date ≥ '2024-01-01'
      AND order_date < '2024-02-01';
```

Q42. EXISTS Clause

Medium

Find employees who have placed at least one order.

```
SELECT e.name
FROM employees e
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.employee_id = e.employee_id
);
```

Q43. NOT EXISTS

Medium

Find employees who have never placed an order.

```
SELECT e.name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.employee_id = e.employee_id
);
```

WHERE & Filtering

Q44. Comparison with Subquery

Medium

Find employees earning more than the average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q45. REGEXP / SIMILAR TO

Medium

Find employees whose name starts with a vowel.

```
-- MySQL
SELECT name FROM employees
WHERE name REGEXP '^[AEIOUaeiou]';

-- PostgreSQL
SELECT name FROM employees
WHERE name SIMILAR TO '[AEIOUaeiou]%';
```

ORDER BY & LIMIT

Sort results and limit output — essential for reports and pagination.

Q46. Order Ascending

Easy

Show all employees sorted by name alphabetically.

```
SELECT name, salary FROM employees
ORDER BY name ASC;
```

Q47. Order Descending

Easy

Show employees sorted by salary highest to lowest.

```
SELECT name, salary FROM employees
ORDER BY salary DESC;
```

Q48. Multiple Sort Columns

Easy

Sort by department ascending, then salary descending within each dept.

```
SELECT name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

Q49. Top 3 Highest Salaries

Easy

Find the 3 highest-paid employees.

```
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 3;
```

ORDER BY & LIMIT

Q50. OFFSET for Pagination

Easy

Get employees 11–20 (page 2, 10 per page).

```
SELECT name, salary FROM employees
ORDER BY employee_id
LIMIT 10 OFFSET 10;
```

Q51. Bottom N Records

Easy

Find the 5 lowest-paid employees.

```
SELECT name, salary FROM employees
ORDER BY salary ASC
LIMIT 5;
```

Q52. Order by Column Position

Easy

Sort results by the 2nd column selected.

```
SELECT name, salary, department
FROM employees
ORDER BY 2 DESC;
```

Q53. Order by Expression

Medium

Sort employees by their annual salary (salary × 12).

```
SELECT name, salary, salary * 12 AS annual
FROM employees
ORDER BY salary * 12 DESC;
```

ORDER BY & LIMIT

Q54. Order by CASE

Medium

Sort so 'Critical' priority comes first, then 'High', then 'Low'.

```
SELECT ticket_id, title, priority
FROM tickets
ORDER BY CASE priority
  WHEN 'Critical' THEN 1
  WHEN 'High' THEN 2
  WHEN 'Medium' THEN 3
  WHEN 'Low' THEN 4
  ELSE 5
END;
```

Q55. NULLS FIRST / LAST

Medium

Sort by bonus descending, putting NULLs at the end.

```
-- PostgreSQL
SELECT name, bonus FROM employees
ORDER BY bonus DESC NULLS LAST;

-- MySQL workaround
SELECT name, bonus FROM employees
ORDER BY bonus IS NULL, bonus DESC;
```

Q56. FETCH FIRST (SQL Standard)

Easy

Get the top 5 employees using SQL standard syntax.

```
SELECT name, salary FROM employees
ORDER BY salary DESC
FETCH FIRST 5 ROWS ONLY;
```

ORDER BY & LIMIT

Q57. Nth Highest Salary

Medium

Find the 3rd highest salary.

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Q58. Random Rows

Easy

Get 5 random employees.

```
-- PostgreSQL
SELECT name FROM employees
ORDER BY RANDOM()
LIMIT 5;

-- MySQL
SELECT name FROM employees
ORDER BY RAND()
LIMIT 5;
```

Q59. Order by Length

Easy

Sort employee names by their length (shortest first).

```
SELECT name, LENGTH(name) AS name_length
FROM employees
ORDER BY LENGTH(name) ASC;
```

ORDER BY & LIMIT

Q60. Pagination with Total Count

Medium

Get page 3 (10 per page) with total count.

```
SELECT name, salary,  
       COUNT(*) OVER() AS total_records  
FROM employees  
ORDER BY employee_id  
LIMIT 10 OFFSET 20;
```

Aggregate Functions

Summarize data using *COUNT*, *SUM*, *AVG*, *MIN*, *MAX* and more.

Q61. COUNT All Rows

Easy

Count total number of employees.

```
SELECT COUNT(*) AS total FROM employees;
```

Q62. COUNT Non-NULL

Easy

Count employees who have a phone number.

```
SELECT COUNT(phone) AS has_phone FROM employees;
```

Q63. COUNT DISTINCT

Easy

How many unique departments exist?

```
SELECT COUNT(DISTINCT department) AS dept_count  
FROM employees;
```

Q64. SUM

Easy

Find total salary expense of the company.

```
SELECT SUM(salary) AS total_salary FROM employees;
```

Q65. AVG

Easy

Find the average salary.

```
SELECT ROUND(AVG(salary), 2) AS avg_salary  
FROM employees;
```

Aggregate Functions

Q66. MIN and MAX

Easy

Find the lowest and highest salary.

```
SELECT MIN(salary) AS lowest,  
       MAX(salary) AS highest  
FROM employees;
```

Q67. SUM with Condition

Easy

Find total salary of IT department only.

```
SELECT SUM(salary) AS it_total  
FROM employees  
WHERE department = 'IT';
```

Q68. Average with ROUND

Easy

Find average order amount rounded to 2 decimals.

```
SELECT ROUND(AVG(amount), 2) AS avg_order  
FROM orders;
```

Q69. Conditional COUNT

Medium

Count how many employees earn above and below 50000.

```
SELECT  
    COUNT(CASE WHEN salary > 50000 THEN 1 END) AS above_50k,  
    COUNT(CASE WHEN salary ≤ 50000 THEN 1 END) AS below_50k  
FROM employees;
```

Aggregate Functions

Q70. SUM with CASE (Pivot-like)

Medium

Show total salary per department as columns.

```
SELECT
    SUM(CASE WHEN department='IT' THEN salary ELSE 0 END) AS it_salary,
    SUM(CASE WHEN department='Sales' THEN salary ELSE 0 END) AS
sales_salary,
    SUM(CASE WHEN department='HR' THEN salary ELSE 0 END) AS hr_salary
FROM employees;
```

Q71. Percentage of Total

Medium

Show each department's salary as % of total company salary.

```
SELECT department,
    SUM(salary) AS dept_salary,
    ROUND(SUM(salary) * 100.0 / (SELECT SUM(salary) FROM employees),
2) AS pct
FROM employees
GROUP BY department
ORDER BY pct DESC;
```

Q72. Variance and Std Dev

Medium

Find salary variance and standard deviation.

```
SELECT ROUND(VARIANCE(salary), 2) AS salary_variance,
    ROUND(STDDEV(salary), 2) AS salary_stddev
FROM employees;
```

Aggregate Functions

Q73. Median Salary

Medium

Find the median salary of employees.

```
-- PostgreSQL
SELECT PERCENTILE_CONT(0.5)
       WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees;
```

Q74. Mode (Most Frequent)

Medium

Find the most common salary.

```
SELECT salary, COUNT(*) AS frequency
FROM employees
GROUP BY salary
ORDER BY frequency DESC
LIMIT 1;
```

Q75. Range

Easy

Find the salary range (max - min).

```
SELECT MAX(salary) - MIN(salary) AS salary_range
FROM employees;
```

Q76. Aggregate with NULL

Easy

Find average bonus (NULLs are ignored by AVG).

```
-- AVG ignores NULLs automatically
SELECT AVG(bonus) AS avg_bonus_with_null_ignored,
       AVG(COALESCE(bonus, 0)) AS avg_bonus_null_as_zero
FROM employees;
```

Aggregate Functions

Q77. COUNT vs COUNT DISTINCT

Easy

Show total orders vs unique customers who ordered.

```
SELECT COUNT(*) AS total_orders,  
       COUNT(DISTINCT customer_id) AS unique_customers  
FROM orders;
```

Q78. Weighted Average

Medium

Find weighted average price based on quantity sold.

```
SELECT ROUND(  
    SUM(price * quantity) / SUM(quantity), 2  
) AS weighted_avg_price  
FROM order_items;
```

Q79. Running Percentage

Medium

Show each product's revenue as a percentage of total.

```
SELECT product_name,  
       revenue,  
       ROUND(revenue * 100.0 / SUM(revenue) OVER(), 2) AS pct_of_total  
FROM products  
ORDER BY pct_of_total DESC;
```

Q80. Cumulative Sum

Medium

Show cumulative salary when sorted by employee_id.

```
SELECT employee_id, name, salary,  
       SUM(salary) OVER(ORDER BY employee_id) AS cumulative_salary  
FROM employees;
```

Aggregate Functions

Q81. Moving Average

Medium

Calculate 3-day moving average of daily sales.

```
SELECT sale_date, amount,
       ROUND(AVG(amount) OVER(
         ORDER BY sale_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ), 2) AS moving_avg_3day
FROM daily_sales;
```

Q82. GROUP_CONCAT / STRING_AGG

Medium

List all employee names per department in one row.

```
-- PostgreSQL
SELECT department,
       STRING_AGG(name, ' ' ORDER BY name) AS employees
FROM employees
GROUP BY department;

-- MySQL
SELECT department,
       GROUP_CONCAT(name ORDER BY name SEPARATOR ' ') AS employees
FROM employees
GROUP BY department;
```

Q83. BOOL_AND / BOOL_OR

Medium

Check if all employees in each dept are active.

```
-- PostgreSQL
SELECT department,
       BOOL_AND(is_active) AS all_active,
       BOOL_OR(is_active) AS any_active
FROM employees
GROUP BY department;
```

Aggregate Functions

Q84. ARRAY_AGG

Medium

Get an array of salaries per department.

```
-- PostgreSQL
SELECT department,
       ARRAY_AGG(salary ORDER BY salary DESC) AS salaries
FROM employees
GROUP BY department;
```

Q85. Multiple Aggregates Together

Easy

Get a complete salary summary.

```
SELECT COUNT(*) AS total_emp,
       SUM(salary) AS total_salary,
       ROUND(AVG(salary), 2) AS avg_salary,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary
FROM employees;
```

GROUP BY & HAVING

Group rows and filter groups — essential for reporting and analytics.

Q86. Basic GROUP BY

Easy

Count employees in each department.

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

Q87. GROUP BY with SUM

Easy

Find total salary per department.

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
ORDER BY total_salary DESC;
```

Q88. GROUP BY with AVG

Easy

Find average salary per department.

```
SELECT department,
       ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department;
```

Q89. GROUP BY Multiple Columns

Easy

Count employees per department and city.

```
SELECT department, city, COUNT(*) AS emp_count
FROM employees
GROUP BY department, city
ORDER BY department, city;
```

GROUP BY & HAVING

Q90. HAVING Clause

Easy

Find departments with more than 5 employees.

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Q91. HAVING with AVG

Easy

Find departments where average salary exceeds 50000.

```
SELECT department, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Q92. WHERE + GROUP BY + HAVING

Medium

Among active employees, find departments with total salary > 200000.

```
SELECT department, SUM(salary) AS total
FROM employees
WHERE is_active = true
GROUP BY department
HAVING SUM(salary) > 200000
ORDER BY total DESC;
```

GROUP BY & HAVING

Q93. GROUP BY with MIN/MAX

Easy

Find the min and max salary in each department.

```
SELECT department,
       MIN(salary) AS min_sal,
       MAX(salary) AS max_sal,
       MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY department;
```

Q94. Monthly Revenue Report

Medium

Find total revenue per month.

```
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS revenue,
       COUNT(*) AS order_count
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

Q95. GROUP BY with Year

Easy

Find total orders per year.

```
SELECT EXTRACT(YEAR FROM order_date) AS year,
       COUNT(*) AS total_orders,
       SUM(amount) AS total_revenue
FROM orders
GROUP BY EXTRACT(YEAR FROM order_date)
ORDER BY year;
```

GROUP BY & HAVING

Q96. Day of Week Analysis

Medium

Find which day of the week gets the most orders.

```
SELECT TO_CHAR(order_date, 'Day') AS day_name,  
       COUNT(*) AS order_count,  
       SUM(amount) AS revenue  
FROM orders  
GROUP BY TO_CHAR(order_date, 'Day'),  
         EXTRACT(DOW FROM order_date)  
ORDER BY EXTRACT(DOW FROM order_date);
```

Q97. Top Department by Headcount

Easy

Find the department with the most employees.

```
SELECT department, COUNT(*) AS emp_count  
FROM employees  
GROUP BY department  
ORDER BY emp_count DESC  
LIMIT 1;
```

Q98. GROUP BY with CASE

Medium

Categorize employees into salary bands and count each.

```
SELECT  
  CASE  
    WHEN salary < 30000 THEN 'Low'  
    WHEN salary BETWEEN 30000 AND 60000 THEN 'Medium'  
    WHEN salary > 60000 THEN 'High'  
  END AS salary_band,  
  COUNT(*) AS emp_count  
FROM employees  
GROUP BY salary_band  
ORDER BY emp_count DESC;
```

GROUP BY & HAVING

Q99. HAVING with COUNT DISTINCT

Medium

Find customers who ordered from 3+ different product categories.

```
SELECT customer_id,  
       COUNT(DISTINCT category) AS categories  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(DISTINCT category) ≥ 3;
```

Q100. GROUPING SETS

Hard

Get employee count by department, by city, and overall total.

```
SELECT department, city, COUNT(*) AS emp_count  
FROM employees  
GROUP BY GROUPING SETS (  
    (department),  
    (city),  
    ()  
)  
ORDER BY department, city;
```

Q101. ROLLUP

Hard

Get hierarchical salary totals: dept → city → grand total.

```
SELECT department, city,  
       SUM(salary) AS total_salary  
FROM employees  
GROUP BY ROLLUP(department, city)  
ORDER BY department, city;
```

GROUP BY & HAVING

Q102. CUBE

Hard

Get all possible grouping combinations of department and city.

```
SELECT department, city,  
       COUNT(*) AS emp_count,  
       SUM(salary) AS total_salary  
FROM employees  
GROUP BY CUBE(department, city)  
ORDER BY department, city;
```

Q103. Group by Date Part

Medium

Find hourly order distribution.

```
SELECT EXTRACT(HOUR FROM order_time) AS hour,  
       COUNT(*) AS order_count  
FROM orders  
GROUP BY EXTRACT(HOUR FROM order_time)  
ORDER BY hour;
```

Q104. HAVING Multiple Conditions

Medium

Find departments with 5+ employees AND avg salary between 40k-70k.

```
SELECT department,  
       COUNT(*) AS emp_count,  
       ROUND(AVG(salary), 2) AS avg_sal  
FROM employees  
GROUP BY department  
HAVING COUNT(*) ≥ 5  
       AND AVG(salary) BETWEEN 40000 AND 70000;
```

GROUP BY & HAVING

Q105. FILTER Clause (PostgreSQL)

Medium

Count active and inactive employees per department in one query.

```
SELECT department,
       COUNT(*) FILTER (WHERE is_active = true) AS active,
       COUNT(*) FILTER (WHERE is_active = false) AS inactive
FROM employees
GROUP BY department;
```

Q106. Departments Above Company Average

Medium

Find departments whose avg salary is above the company average.

```
SELECT department, ROUND(AVG(salary), 2) AS dept_avg
FROM employees
GROUP BY department
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Q107. Consecutive Date Grouping

Hard

Find the count of orders per week.

```
SELECT DATE_TRUNC('week', order_date) AS week_start,
       COUNT(*) AS orders,
       SUM(amount) AS revenue
FROM orders
GROUP BY DATE_TRUNC('week', order_date)
ORDER BY week_start;
```

GROUP BY & HAVING

Q108. Quartile Distribution

Hard

Divide employees into 4 salary quartiles and count each.

```
SELECT quartile, COUNT(*) AS emp_count,  
       MIN(salary) AS min_sal, MAX(salary) AS max_sal  
FROM (  
    SELECT salary, NTILE(4) OVER(ORDER BY salary) AS quartile  
    FROM employees  
) sub  
GROUP BY quartile  
ORDER BY quartile;
```

Q109. Year-over-Year Growth

Hard

Calculate revenue growth percentage year over year.

```
WITH yearly AS (  
    SELECT EXTRACT(YEAR FROM order_date) AS year,  
           SUM(amount) AS revenue  
    FROM orders  
    GROUP BY EXTRACT(YEAR FROM order_date)  
)  
SELECT year, revenue,  
       LAG(revenue) OVER(ORDER BY year) AS prev_year,  
       ROUND((revenue - LAG(revenue) OVER(ORDER BY year))  
             * 100.0 / LAG(revenue) OVER(ORDER BY year), 2) AS  
growth_pct  
FROM yearly  
ORDER BY year;
```

GROUP BY & HAVING

Q110. Top N per Group

Hard

Find the top 2 highest-paid employees in each department.

```
SELECT department, name, salary
FROM (
    SELECT department, name, salary,
           ROW_NUMBER() OVER(
               PARTITION BY department ORDER BY salary DESC
           ) AS rn
    FROM employees
) ranked
WHERE rn ≤ 2
ORDER BY department, salary DESC;
```

JOINS

Combine data from multiple tables — the heart of relational databases.

Q111. INNER JOIN

Easy

Show each employee with their department name.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Q112. LEFT JOIN

Easy

Show ALL employees, even those without a department.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Q113. RIGHT JOIN

Easy

Show ALL departments, even those with no employees.

```
SELECT d.department_name, e.name
FROM employees e
RIGHT JOIN departments d
    ON e.department_id = d.department_id;
```

JOINS

Q114. FULL OUTER JOIN

Medium

Show all employees and all departments, including unmatched.

```
SELECT e.name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
    ON e.department_id = d.department_id;
```

Q115. CROSS JOIN

Easy

Generate all possible employee-project combinations.

```
SELECT e.name, p.project_name
FROM employees e
CROSS JOIN projects p;
```

Q116. Self JOIN — Manager

Medium

Show each employee with their manager's name.

```
SELECT e.name AS employee,
       m.name AS manager
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.employee_id;
```

Q117. JOIN with WHERE

Easy

Show orders with customer names for orders above 1000.

```
SELECT o.order_id, c.customer_name, o.amount
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
WHERE o.amount > 1000;
```

JOINS

Q118. JOIN Three Tables

Medium

Show order details: customer name, product name, quantity.

```
SELECT c.customer_name,  
       p.product_name,  
       oi.quantity,  
       oi.price  
FROM order_items oi  
INNER JOIN orders o ON oi.order_id = o.order_id  
INNER JOIN customers c ON o.customer_id = c.customer_id  
INNER JOIN products p ON oi.product_id = p.product_id;
```

Q119. LEFT JOIN — Find Unmatched

Medium

Find customers who have never placed an order.

```
SELECT c.customer_name  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

Q120. JOIN with Aggregation

Medium

Find total order amount per customer.

```
SELECT c.customer_name,  
       COUNT(o.order_id) AS total_orders,  
       SUM(o.amount) AS total_spent  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_name  
ORDER BY total_spent DESC;
```

JOINS

Q121. JOIN with Date Filter

Medium

Get employee names who joined in the last 6 months with their dept.

```
SELECT e.name, d.department_name, e.join_date
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
WHERE e.join_date ≥ CURRENT_DATE - INTERVAL '6 months';
```

Q122. Multiple JOINS with GROUP BY

Medium

Find total sales per product category.

```
SELECT c.category_name,
       SUM(oi.quantity * oi.price) AS total_sales
FROM order_items oi
INNER JOIN products p ON oi.product_id = p.product_id
INNER JOIN categories c ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_sales DESC;
```

Q123. Self JOIN — Employees Same Dept

Medium

Find pairs of employees in the same department.

```
SELECT e1.name AS employee1,
       e2.name AS employee2,
       e1.department
FROM employees e1
INNER JOIN employees e2
    ON e1.department = e2.department
    AND e1.employee_id < e2.employee_id;
```

JOINS

Q124. NATURAL JOIN

Easy

Join employees and departments on common column names.

```
SELECT name, department_name
FROM employees
NATURAL JOIN departments;
-- Joins on ALL columns with same name - use with caution!
```

Q125. JOIN with USING

Easy

Join when both tables have the same column name.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d USING (department_id);
```

Q126. Anti-JOIN Pattern

Medium

Find products that have never been ordered.

```
-- Method 1: LEFT JOIN + IS NULL
SELECT p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;

-- Method 2: NOT EXISTS
SELECT p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi
    WHERE oi.product_id = p.product_id
);
```

JOINS

Q127. Semi-JOIN Pattern

Medium

Find departments that have at least one employee.

```
SELECT DISTINCT d.department_name
FROM departments d
INNER JOIN employees e ON d.department_id = e.department_id;

-- Better with EXISTS:
SELECT d.department_name FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e
    WHERE e.department_id = d.department_id
);
```

Q128. JOIN with COALESCE

Medium

Show employees with dept name; if no dept, show 'Unassigned'.

```
SELECT e.name,
       COALESCE(d.department_name, 'Unassigned') AS department
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id;
```

Q129. Many-to-Many JOIN

Medium

Show students and their enrolled courses (via junction table).

```
SELECT s.student_name, c.course_name
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id
ORDER BY s.student_name;
```

JOINS

Q130. JOIN with Subquery

Medium

Show employees with their department's average salary.

```
SELECT e.name, e.salary, dept_avg.avg_salary
FROM employees e
INNER JOIN (
    SELECT department_id,
           ROUND(AVG(salary), 2) AS avg_salary
    FROM employees
    GROUP BY department_id
) dept_avg ON e.department_id = dept_avg.department_id;
```

Q131. LATERAL JOIN

Hard

For each department, get the top 3 highest-paid employees.

```
-- PostgreSQL
SELECT d.department_name, top3.name, top3.salary
FROM departments d
CROSS JOIN LATERAL (
    SELECT name, salary
    FROM employees e
    WHERE e.department_id = d.department_id
    ORDER BY salary DESC
    LIMIT 3
) top3;
```

JOINS

Q132. Self JOIN — Salary Comparison

Medium

Find employees who earn more than their manager.

```
SELECT e.name AS employee,  
       e.salary AS emp_salary,  
       m.name AS manager,  
       m.salary AS mgr_salary  
FROM employees e  
INNER JOIN employees m ON e.manager_id = m.employee_id  
WHERE e.salary > m.salary;
```

Q133. JOIN Four Tables

Medium

Full order report: customer, order, product, category.

```
SELECT c.customer_name, o.order_date,  
       p.product_name, cat.category_name,  
       oi.quantity, oi.price  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN products p ON oi.product_id = p.product_id  
JOIN categories cat ON p.category_id = cat.category_id  
ORDER BY o.order_date DESC;
```

JOINS

Q134. JOIN with Date Overlap

Hard

Find employees who were on leave on a specific project's dates.

```
SELECT e.name, l.leave_start, l.leave_end,
       p.project_name, p.start_date, p.end_date
FROM employees e
JOIN leaves l ON e.employee_id = l.employee_id
JOIN project_assignments pa ON e.employee_id = pa.employee_id
JOIN projects p ON pa.project_id = p.project_id
WHERE l.leave_start ≤ p.end_date
      AND l.leave_end ≥ p.start_date;
```

Q135. Recursive Self JOIN — Hierarchy

Hard

Show the full management chain for each employee.

```
WITH RECURSIVE hierarchy AS (
  SELECT employee_id, name, manager_id, 1 AS level,
         name::TEXT AS chain
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  SELECT e.employee_id, e.name, e.manager_id, h.level + 1,
         h.chain || ' → ' || e.name
  FROM employees e
  INNER JOIN hierarchy h ON e.manager_id = h.employee_id
)
SELECT name, level, chain FROM hierarchy
ORDER BY chain;
```

JOINS

Q136. UNION with JOINS

Medium

Get a combined list of customers and suppliers with their cities.

```
SELECT name, city, 'Customer' AS type
FROM customers
UNION ALL
SELECT name, city, 'Supplier' AS type
FROM suppliers
ORDER BY city, name;
```

Q137. INTERSECT

Medium

Find cities that have both customers and suppliers.

```
SELECT city FROM customers
INTERSECT
SELECT city FROM suppliers;
```

Q138. EXCEPT

Medium

Find customers who are in cities where there are no suppliers.

```
SELECT city FROM customers
EXCEPT
SELECT city FROM suppliers;
```

Q139. Non-Equi JOIN

Hard

Find employees whose salary falls within a grade range.

```
SELECT e.name, e.salary, g.grade
FROM employees e
INNER JOIN salary_grades g
ON e.salary BETWEEN g.min_salary AND g.max_salary;
```

JOINS

Q140. Conditional JOIN

Hard

Join to different tables based on a type column.

```
SELECT o.order_id,  
       COALESCE(c.name, s.name) AS entity_name  
FROM orders o  
LEFT JOIN customers c  
  ON o.entity_type = 'customer' AND o.entity_id = c.customer_id  
LEFT JOIN suppliers s  
  ON o.entity_type = 'supplier' AND o.entity_id = s.supplier_id;
```

Q141. JOIN with Window Function

Hard

Rank employees within each department by salary.

```
SELECT d.department_name, e.name, e.salary,  
       RANK() OVER(  
         PARTITION BY d.department_id ORDER BY e.salary DESC  
       ) AS dept_rank  
FROM employees e  
INNER JOIN departments d ON e.department_id = d.department_id;
```

Q142. Closest Match JOIN

Hard

For each order, find the most recent price before the order date.

```
SELECT DISTINCT ON (o.order_id)  
       o.order_id, o.product_id, o.order_date,  
       ph.price, ph.effective_date  
FROM orders o  
JOIN price_history ph  
  ON o.product_id = ph.product_id  
  AND ph.effective_date ≤ o.order_date  
ORDER BY o.order_id, ph.effective_date DESC;
```

JOINS

Q143. Employees Without Orders (Multiple Methods)

Medium

Find employees who haven't placed any orders — three ways.

```
-- Method 1: LEFT JOIN
SELECT e.name FROM employees e
LEFT JOIN orders o ON e.employee_id = o.employee_id
WHERE o.order_id IS NULL;

-- Method 2: NOT IN
SELECT name FROM employees
WHERE employee_id NOT IN (
    SELECT DISTINCT employee_id FROM orders
    WHERE employee_id IS NOT NULL
);

-- Method 3: NOT EXISTS
SELECT name FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.employee_id = e.employee_id
);
```

Q144. Pivoting with JOINS

Hard

Show each student's marks in Math, Science, English as columns.

```
SELECT s.student_name,
       MAX(CASE WHEN sub.subject_name = 'Math' THEN m.marks END) AS
math,
       MAX(CASE WHEN sub.subject_name = 'Science' THEN m.marks END) AS
science,
       MAX(CASE WHEN sub.subject_name = 'English' THEN m.marks END) AS
english
FROM students s
JOIN marks m ON s.student_id = m.student_id
JOIN subjects sub ON m.subject_id = sub.subject_id
GROUP BY s.student_name;
```

JOINS

Q145. Full Report with Multiple JOINS

Hard

Complete employee report with department, manager, and project count.

```
SELECT e.name,  
       d.department_name,  
       m.name AS manager_name,  
       COUNT(DISTINCT pa.project_id) AS project_count,  
       e.salary  
FROM employees e  
LEFT JOIN departments d ON e.department_id = d.department_id  
LEFT JOIN employees m ON e.manager_id = m.employee_id  
LEFT JOIN project_assignments pa ON e.employee_id = pa.employee_id  
GROUP BY e.name, d.department_name, m.name, e.salary  
ORDER BY e.name;
```

Subqueries

Queries within queries — for complex filtering, derived tables, and computed values.

Q146. Scalar Subquery in WHERE

Easy

Find employees earning above the average salary.

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q147. Subquery with IN

Easy

Find employees in departments located in 'Mumbai'.

```
SELECT name, department_id FROM employees
WHERE department_id IN (
    SELECT department_id FROM departments
    WHERE city = 'Mumbai'
);
```

Q148. Subquery in SELECT

Medium

Show each employee with the company's total employee count.

```
SELECT name, salary,
       (SELECT COUNT(*) FROM employees) AS total_employees
FROM employees;
```

Subqueries

Q149. Correlated Subquery

Medium

Find employees who earn the highest salary in their department.

```
SELECT name, department, salary
FROM employees e1
WHERE salary = (
    SELECT MAX(salary) FROM employees e2
    WHERE e2.department = e1.department
);
```

Q150. Subquery in FROM (Derived Table)

Medium

Get department-wise stats and filter departments with high avg salary.

```
SELECT * FROM (
    SELECT department,
        COUNT(*) AS emp_count,
        ROUND(AVG(salary), 2) AS avg_salary
    FROM employees
    GROUP BY department
) dept_stats
WHERE avg_salary > 50000;
```

Q151. Multi-Row Subquery with ALL

Medium

Find employees earning more than ALL employees in HR department.

```
SELECT name, salary FROM employees
WHERE salary > ALL (
    SELECT salary FROM employees WHERE department = 'HR'
);
```

Subqueries

Q152. Multi-Row Subquery with ANY

Medium

Find employees earning more than ANY employee in Sales.

```
SELECT name, salary FROM employees
WHERE salary > ANY (
    SELECT salary FROM employees WHERE department = 'Sales'
);
```

Q153. Nested Subqueries

Hard

Find the department name of the highest-paid employee.

```
SELECT department_name FROM departments
WHERE department_id = (
    SELECT department_id FROM employees
    WHERE salary = (SELECT MAX(salary) FROM employees)
);
```

Q154. Subquery with EXISTS

Medium

Find departments that have at least one high earner (> 80000).

```
SELECT department_name FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e
    WHERE e.department_id = d.department_id
    AND e.salary > 80000
);
```

Subqueries

Q155. CTE (Common Table Expression)

Medium

Use CTE to find departments above average salary.

```
WITH dept_avg AS (  
    SELECT department,  
           ROUND(AVG(salary), 2) AS avg_salary  
    FROM employees  
    GROUP BY department  
)  
SELECT * FROM dept_avg  
WHERE avg_salary > (SELECT AVG(salary) FROM employees);
```

Q156. Multiple CTEs

Medium

Compare department revenue with overall company average.

```
WITH dept_revenue AS (  
    SELECT department, SUM(salary) AS total  
    FROM employees GROUP BY department  
)  
,  
company_avg AS (  
    SELECT AVG(total) AS avg_total FROM dept_revenue  
)  
SELECT d.department, d.total,  
       c.avg_total,  
       CASE WHEN d.total > c.avg_total THEN 'Above' ELSE 'Below' END AS  
status  
FROM dept_revenue d, company_avg c  
ORDER BY d.total DESC;
```

Subqueries

Q157. Recursive CTE — Numbers

Medium

Generate numbers from 1 to 10 using recursive CTE.

```
WITH RECURSIVE nums AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1 FROM nums WHERE n < 10  
)  
SELECT n FROM nums;
```

Q158. Recursive CTE — Date Series

Medium

Generate all dates in January 2024.

```
WITH RECURSIVE dates AS (  
    SELECT DATE '2024-01-01' AS d  
    UNION ALL  
    SELECT d + INTERVAL '1 day' FROM dates  
    WHERE d < '2024-01-31'  
)  
SELECT d::DATE AS date FROM dates;
```

Q159. Subquery for Row Numbering

Medium

Delete duplicate rows keeping only the first occurrence.

```
DELETE FROM employees  
WHERE employee_id NOT IN (  
    SELECT MIN(employee_id)  
    FROM employees  
    GROUP BY name, department, salary  
);
```

Subqueries

Q160. Correlated Subquery — Running Total

Hard

Show each order with a running total of all previous orders.

```
SELECT order_id, order_date, amount,
       (SELECT SUM(amount) FROM orders o2
        WHERE o2.order_date ≤ o1.order_date) AS running_total
FROM orders o1
ORDER BY order_date;
```

Q161. Subquery to Find Duplicates

Medium

Find employees with duplicate email addresses.

```
SELECT * FROM employees
WHERE email IN (
    SELECT email FROM employees
    GROUP BY email
    HAVING COUNT(*) > 1
)
ORDER BY email;
```

Q162. CTE for Percentile Rank

Hard

Show each employee's salary percentile within their department.

```
WITH ranked AS (
    SELECT name, department, salary,
           PERCENT_RANK() OVER(
               PARTITION BY department ORDER BY salary
           ) AS percentile
    FROM employees
)
SELECT name, department, salary,
       ROUND(percentile * 100, 1) AS percentile_rank
FROM ranked
ORDER BY department, percentile DESC;
```

Subqueries

Q163. Inline View for Top Per Group

Hard

Find the most recent order for each customer.

```
SELECT c.customer_name, o.order_id, o.order_date, o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date = (
    SELECT MAX(o2.order_date) FROM orders o2
    WHERE o2.customer_id = c.customer_id
);
```

Q164. CTE with UPDATE

Hard

Give a 10% raise to employees below their department average.

```
WITH dept_avg AS (
    SELECT department_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department_id
)
UPDATE employees e
SET salary = salary * 1.10
FROM dept_avg d
WHERE e.department_id = d.department_id
AND e.salary < d.avg_sal;
```

Q165. Recursive CTE — Fibonacci

Hard

Generate the first 10 Fibonacci numbers.

```
WITH RECURSIVE fib AS (
    SELECT 1 AS n, 0::BIGINT AS a, 1::BIGINT AS b
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < 10
)
SELECT n, a AS fibonacci FROM fib;
```

Subqueries

Q166. Subquery — Employees Without Max Salary

Easy

Find employees who do NOT have the highest salary.

```
SELECT name, salary FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q167. CTE — Month-over-Month Comparison

Hard

Compare this month's revenue with last month.

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS month,
           SUM(amount) AS revenue
    FROM orders
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month,
       revenue,
       LAG(revenue) OVER(ORDER BY month) AS prev_month,
       revenue - LAG(revenue) OVER(ORDER BY month) AS change,
       ROUND((revenue - LAG(revenue) OVER(ORDER BY month))
             * 100.0 / LAG(revenue) OVER(ORDER BY month), 2) AS
pct_change
FROM monthly
ORDER BY month;
```

Q168. Subquery — Second Highest Salary

Medium

Find the second highest salary using subquery.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Subqueries

Q169. CTE — Gap Analysis

Hard

Find missing dates (gaps) in an order sequence.

```
WITH RECURSIVE date_range AS (  
    SELECT MIN(order_date) AS d FROM orders  
    UNION ALL  
    SELECT d + INTERVAL '1 day' FROM date_range  
    WHERE d < (SELECT MAX(order_date) FROM orders)  
)  
order_dates AS (  
    SELECT DISTINCT order_date FROM orders  
)  
SELECT dr.d::DATE AS missing_date  
FROM date_range dr  
LEFT JOIN order_dates od ON dr.d::DATE = od.order_date  
WHERE od.order_date IS NULL  
ORDER BY dr.d;
```

Q170. CTE — Materialized (PostgreSQL 12+)

Hard

Force PostgreSQL to materialize a CTE for performance.

```
WITH expensive_calc AS MATERIALIZED (  
    SELECT department_id,  
           AVG(salary) AS avg_sal,  
           COUNT(*) AS cnt  
    FROM employees  
    GROUP BY department_id  
)  
SELECT e.name, e.salary, ec.avg_sal  
FROM employees e  
JOIN expensive_calc ec ON e.department_id = ec.department_id  
WHERE e.salary > ec.avg_sal;
```

String Functions

Manipulate and analyze text data within SQL.

Q171. UPPER and LOWER

Easy

Convert name to uppercase and email to lowercase.

```
SELECT UPPER(name) AS upper_name,  
       LOWER(email) AS lower_email  
FROM employees;
```

Q172. LENGTH

Easy

Find employees whose name is longer than 15 characters.

```
SELECT name, LENGTH(name) AS name_len  
FROM employees  
WHERE LENGTH(name) > 15;
```

Q173. SUBSTRING / SUBSTR

Easy

Extract the first 3 characters of each employee's name.

```
SELECT name,  
       SUBSTRING(name FROM 1 FOR 3) AS initials  
FROM employees;
```

Q174. TRIM, LTRIM, RTRIM

Easy

Remove leading/trailing spaces from names.

```
SELECT TRIM(name) AS trimmed,  
       LTRIM(name) AS left_trimmed,  
       RTRIM(name) AS right_trimmed  
FROM employees;
```

String Functions

Q175. REPLACE

Easy

Replace all spaces in phone numbers with dashes.

```
SELECT phone,
       REPLACE(phone, ' ', '-') AS formatted_phone
FROM employees;
```

Q176. POSITION / STRPOS

Easy

Find the position of '@' in email addresses.

```
-- PostgreSQL
SELECT email, POSITION('@' IN email) AS at_pos
FROM employees;

-- MySQL
SELECT email, LOCATE('@', email) AS at_pos
FROM employees;
```

Q177. LEFT and RIGHT

Easy

Get first 5 and last 4 characters of a phone number.

```
SELECT phone,
       LEFT(phone, 5) AS area_code,
       RIGHT(phone, 4) AS last_four
FROM employees;
```

Q178. LPAD and RPAD

Easy

Pad employee IDs with zeros to 6 digits.

```
SELECT employee_id,
       LPAD(employee_id::TEXT, 6, '0') AS padded_id
FROM employees;
```

String Functions

Q179. INITCAP

Easy

Capitalize the first letter of each word in the name.

```
SELECT INITCAP(name) AS proper_name
FROM employees;
```

Q180. REVERSE

Easy

Reverse each employee's name.

```
SELECT name, REVERSE(name) AS reversed
FROM employees;
```

Q181. REPEAT

Easy

Create a simple bar chart using repeated characters.

```
SELECT department,
       COUNT(*) AS cnt,
       REPEAT('█', COUNT(*)) AS bar
FROM employees
GROUP BY department
ORDER BY cnt DESC;
```

Q182. SPLIT_PART

Medium

Extract the domain from email addresses.

```
-- PostgreSQL
SELECT email,
       SPLIT_PART(email, '@', 2) AS domain
FROM employees;
```

String Functions

Q183. REGEXP_REPLACE

Medium

Remove all non-numeric characters from phone numbers.

```
SELECT phone,  
       REGEXP_REPLACE(phone, '[^0-9]', '', 'g') AS digits_only  
FROM employees;
```

Q184. REGEXP_MATCHES

Medium

Extract all numbers from a text column.

```
-- PostgreSQL  
SELECT description,  
       REGEXP_MATCHES(description, '(\d+)', 'g') AS numbers  
FROM products;
```

Q185. TRANSLATE

Medium

Replace vowels with asterisks in names.

```
-- PostgreSQL  
SELECT name,  
       TRANSLATE(name, 'aeiouAEIOU', '*****') AS masked  
FROM employees;
```

String Functions

Q186. Format as Currency

Medium

Display salary in Indian format with commas.

```
-- PostgreSQL
SELECT name,
       TO_CHAR(salary, '99,99,999') AS formatted_salary
FROM employees;

-- MySQL
SELECT name,
       FORMAT(salary, 0) AS formatted_salary
FROM employees;
```

Q187. Mask Sensitive Data

Medium

Show only last 4 digits of phone, mask the rest.

```
SELECT name,
       REPEAT('*', LENGTH(phone) - 4) || RIGHT(phone, 4) AS masked_phone
FROM employees;
```

Q188. CONCAT_WS

Easy

Combine address fields with comma separator, skipping NULLs.

```
SELECT CONCAT_WS(', ', street, city, state, zip) AS full_address
FROM employees;
```

String Functions

Q189. ASCII and CHR

Easy

Get ASCII code of first character and convert code to character.

```
SELECT name,  
       ASCII(LEFT(name, 1)) AS first_char_ascii,  
       CHR(65) AS char_from_65  
FROM employees;
```

Q190. Extract Email Username

Medium

Get the username part (before @) from emails.

```
SELECT email,  
       SUBSTRING(email FROM 1 FOR POSITION('@' IN email) - 1) AS  
username  
FROM employees;
```

Date & Time Functions

Work with dates and timestamps for time-based analysis and reporting.

Q191. Current Date and Time

Easy

Get the current date, time, and timestamp.

```
SELECT CURRENT_DATE AS today,  
       CURRENT_TIME AS time_now,  
       CURRENT_TIMESTAMP AS full_timestamp,  
       NOW() AS also_now;
```

Q192. EXTRACT Parts

Easy

Extract year, month, day from a date.

```
SELECT order_date,  
       EXTRACT(YEAR FROM order_date) AS year,  
       EXTRACT(MONTH FROM order_date) AS month,  
       EXTRACT(DAY FROM order_date) AS day  
FROM orders;
```

Q193. DATE_TRUNC

Easy

Truncate timestamp to the beginning of the month.

```
SELECT order_date,  
       DATE_TRUNC('month', order_date) AS month_start,  
       DATE_TRUNC('year', order_date) AS year_start,  
       DATE_TRUNC('week', order_date) AS week_start  
FROM orders;
```

Date & Time Functions

Q194. Date Arithmetic

Easy

Find the date 30 days from now and 90 days ago.

```
SELECT CURRENT_DATE AS today,  
       CURRENT_DATE + INTERVAL '30 days' AS thirty_days_later,  
       CURRENT_DATE - INTERVAL '90 days' AS ninety_days_ago;
```

Q195. Date Difference

Easy

Find how many days since each employee joined.

```
-- PostgreSQL  
SELECT name, join_date,  
       CURRENT_DATE - join_date AS days_employed  
FROM employees;  
  
-- MySQL  
SELECT name, join_date,  
       DATEDIFF(CURDATE(), join_date) AS days_employed  
FROM employees;
```

Q196. AGE Function

Easy

Find each employee's tenure in years and months.

```
-- PostgreSQL  
SELECT name, join_date,  
       AGE(CURRENT_DATE, join_date) AS tenure  
FROM employees;
```

Date & Time Functions

Q197. TO_CHAR for Date Formatting

Easy

Format order dates in different styles.

```
SELECT order_date,  
       TO_CHAR(order_date, 'DD-Mon-YYYY') AS formatted1,  
       TO_CHAR(order_date, 'Day, DD Month YYYY') AS formatted2,  
       TO_CHAR(order_date, 'YYYY/MM/DD') AS formatted3  
FROM orders;
```

Q198. TO_DATE — String to Date

Easy

Convert a string '15-Jan-2024' to a date.

```
SELECT TO_DATE('15-Jan-2024', 'DD-Mon-YYYY') AS parsed_date;
```

Q199. Orders in Last 7 Days

Easy

Find all orders from the last 7 days.

```
SELECT * FROM orders  
WHERE order_date ≥ CURRENT_DATE - INTERVAL '7 days';
```

Q200. Group by Month Name

Medium

Show order count per month name.

```
SELECT TO_CHAR(order_date, 'Month') AS month_name,  
       EXTRACT(MONTH FROM order_date) AS month_num,  
       COUNT(*) AS orders  
FROM orders  
GROUP BY TO_CHAR(order_date, 'Month'),  
         EXTRACT(MONTH FROM order_date)  
ORDER BY month_num;
```

Date & Time Functions

Q201. Business Days Calculation

Hard

Count business days (Mon-Fri) between two dates.

```
WITH RECURSIVE dates AS (  
    SELECT DATE '2024-01-01' AS d  
    UNION ALL  
    SELECT d + 1 FROM dates WHERE d < '2024-01-31'  
)  
SELECT COUNT(*) AS business_days  
FROM dates  
WHERE EXTRACT(DOW FROM d) NOT IN (0, 6);
```

Q202. EPOCH — Timestamps to Seconds

Medium

Calculate order processing time in hours.

```
SELECT order_id,  
       EXTRACT(EPOCH FROM (shipped_at - ordered_at)) / 3600  
       AS processing_hours  
FROM orders  
WHERE shipped_at IS NOT NULL;
```

Q203. GENERATE_SERIES for Dates

Medium

Generate all dates in a month even if no orders exist.

```
-- PostgreSQL  
SELECT d::DATE AS date,  
       COALESCE(o.order_count, 0) AS orders  
FROM GENERATE_SERIES('2024-01-01', '2024-01-31', '1 day'::INTERVAL) d  
LEFT JOIN (  
    SELECT order_date, COUNT(*) AS order_count  
    FROM orders GROUP BY order_date  
) o ON d::DATE = o.order_date  
ORDER BY d;
```

Date & Time Functions

Q204. MAKE_DATE / MAKE_TIMESTAMP

Easy

Construct a date from year, month, day columns.

```
-- PostgreSQL
SELECT MAKE_DATE(2024, 6, 15) AS constructed_date;
SELECT MAKE_TIMESTAMP(2024, 6, 15, 10, 30, 0) AS constructed_ts;
```

Q205. Date Overlap Check

Hard

Find overlapping leave requests for the same employee.

```
SELECT l1.leave_id, l2.leave_id,
       l1.start_date, l1.end_date,
       l2.start_date, l2.end_date
FROM leaves l1
JOIN leaves l2
  ON l1.employee_id = l2.employee_id
 AND l1.leave_id < l2.leave_id
 AND l1.start_date ≤ l2.end_date
 AND l1.end_date ≥ l2.start_date;
```

Q206. Employees Joined This Month

Easy

List employees who joined in the current month.

```
SELECT name, join_date FROM employees
WHERE DATE_TRUNC('month', join_date)
      = DATE_TRUNC('month', CURRENT_DATE);
```

Date & Time Functions

Q207. Quarter-wise Report

Medium

Summarize revenue by fiscal quarter.

```
SELECT EXTRACT(YEAR FROM order_date) AS year,  
       EXTRACT(QUARTER FROM order_date) AS quarter,  
       'Q' || EXTRACT(QUARTER FROM order_date) AS label,  
       SUM(amount) AS revenue  
FROM orders  
GROUP BY year, quarter  
ORDER BY year, quarter;
```

Q208. Time Zones

Medium

Convert timestamps between time zones.

```
SELECT created_at,  
       created_at AT TIME ZONE 'UTC' AS utc_time,  
       created_at AT TIME ZONE 'Asia/Kolkata' AS ist_time  
FROM orders;
```

Date & Time Functions

Q209. Consecutive Days Streak

Hard

Find the longest consecutive login streak per user.

```
WITH grouped AS (  
    SELECT user_id, login_date,  
           login_date - ROW_NUMBER() OVER(  
               PARTITION BY user_id ORDER BY login_date  
           )::INT AS grp  
    FROM logins  
)  
SELECT user_id,  
       MIN(login_date) AS streak_start,  
       MAX(login_date) AS streak_end,  
       COUNT(*) AS streak_days  
FROM grouped  
GROUP BY user_id, grp  
ORDER BY streak_days DESC;
```

Q210. Working Hours Calculation

Hard

Calculate total working hours logged this week per employee.

```
SELECT e.name,  
       SUM(EXTRACT(EPOCH FROM (t.clock_out - t.clock_in)) / 3600)  
       AS total_hours  
FROM time_logs t  
JOIN employees e ON t.employee_id = e.employee_id  
WHERE t.log_date ≥ DATE_TRUNC('week', CURRENT_DATE)  
GROUP BY e.name  
ORDER BY total_hours DESC;
```

CASE & Conditional Logic

Add if-then-else logic directly in your SQL queries.

Q211. Simple CASE

Easy

Categorize employees by salary level.

```
SELECT name, salary,  
       CASE  
         WHEN salary < 30000 THEN 'Junior'  
         WHEN salary BETWEEN 30000 AND 60000 THEN 'Mid-Level'  
         WHEN salary > 60000 THEN 'Senior'  
       END AS level  
FROM employees;
```

Q212. CASE in ORDER BY

Medium

Custom sort: Directors first, then Managers, then Staff.

```
SELECT name, role FROM employees  
ORDER BY CASE role  
  WHEN 'Director' THEN 1  
  WHEN 'Manager' THEN 2  
  WHEN 'Staff' THEN 3  
  ELSE 4  
END;
```

Q213. CASE in WHERE

Medium

Dynamic filtering: if dept is IT, salary > 50k; else salary > 40k.

```
SELECT name, department, salary FROM employees  
WHERE salary > CASE  
  WHEN department = 'IT' THEN 50000  
  ELSE 40000  
END;
```

CASE & Conditional Logic

Q214. CASE for Pivoting

Medium

Show monthly revenue as columns (Jan, Feb, Mar...).

```
SELECT EXTRACT(YEAR FROM order_date) AS year,  
       SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 1 THEN amount ELSE 0  
END) AS jan,  
       SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 2 THEN amount ELSE 0  
END) AS feb,  
       SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 3 THEN amount ELSE 0  
END) AS mar,  
       SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 4 THEN amount ELSE 0  
END) AS apr  
FROM orders  
GROUP BY EXTRACT(YEAR FROM order_date);
```

Q215. Nested CASE

Medium

Calculate bonus: IT gets 15%, Sales gets 20%, others get 10%; seniors get extra 5%.

```
SELECT name, department, salary,  
       CASE department  
         WHEN 'IT' THEN salary * 0.15  
         WHEN 'Sales' THEN salary * 0.20  
         ELSE salary * 0.10  
       END +  
       CASE WHEN salary > 60000 THEN salary * 0.05 ELSE 0 END  
       AS bonus  
FROM employees;
```

CASE & Conditional Logic

Q216. CASE for NULL Handling

Easy

Show 'No Phone' if phone is NULL, else show the phone number.

```
SELECT name,  
       CASE  
         WHEN phone IS NULL THEN 'No Phone'  
         ELSE phone  
       END AS phone_display  
FROM employees;
```

Q217. CASE for Status Labels

Easy

Convert numeric status to readable text.

```
SELECT order_id,  
       CASE status  
         WHEN 1 THEN 'Pending'  
         WHEN 2 THEN 'Processing'  
         WHEN 3 THEN 'Shipped'  
         WHEN 4 THEN 'Delivered'  
         WHEN 5 THEN 'Cancelled'  
         ELSE 'Unknown'  
       END AS status_text  
FROM orders;
```

Q218. CASE for Conditional Aggregation

Medium

Count orders by status per month.

```
SELECT DATE_TRUNC('month', order_date) AS month,  
       COUNT(CASE WHEN status = 'Delivered' THEN 1 END) AS delivered,  
       COUNT(CASE WHEN status = 'Cancelled' THEN 1 END) AS cancelled,  
       COUNT(CASE WHEN status = 'Pending' THEN 1 END) AS pending  
FROM orders  
GROUP BY DATE_TRUNC('month', order_date)  
ORDER BY month;
```

CASE & Conditional Logic

Q219. CASE for Grade Assignment

Easy

Assign letter grades based on marks.

```
SELECT student_name, marks,
       CASE
         WHEN marks ≥ 90 THEN 'A+'
         WHEN marks ≥ 80 THEN 'A'
         WHEN marks ≥ 70 THEN 'B'
         WHEN marks ≥ 60 THEN 'C'
         WHEN marks ≥ 50 THEN 'D'
         ELSE 'F'
       END AS grade
FROM students;
```

Q220. CASE for Safe Division

Medium

Calculate conversion rate, avoiding division by zero.

```
SELECT campaign_name, clicks, conversions,
       CASE
         WHEN clicks = 0 THEN 0
         ELSE ROUND(conversions * 100.0 / clicks, 2)
       END AS conversion_rate
FROM campaigns;
```

Q221. CASE in UPDATE

Medium

Give different raise percentages based on department.

```
UPDATE employees
SET salary = salary * CASE department
  WHEN 'IT' THEN 1.12
  WHEN 'Sales' THEN 1.10
  WHEN 'HR' THEN 1.08
  ELSE 1.05
END;
```

CASE & Conditional Logic

Q222. CASE with BETWEEN

Easy

Classify products by price range.

```
SELECT product_name, price,  
       CASE  
         WHEN price BETWEEN 0 AND 100 THEN 'Budget'  
         WHEN price BETWEEN 101 AND 500 THEN 'Standard'  
         WHEN price BETWEEN 501 AND 2000 THEN 'Premium'  
         ELSE 'Luxury'  
       END AS price_tier  
FROM products;
```

Q223. CASE for Boolean Display

Easy

Show 'Yes'/'No' instead of true/false for is_active.

```
SELECT name,  
       CASE WHEN is_active THEN 'Yes' ELSE 'No' END AS active_status  
FROM employees;
```

CASE & Conditional Logic

Q224. CASE for Dynamic Column Selection

Hard

Show salary in local currency based on country.

```
SELECT name, country, salary,
       CASE country
         WHEN 'India' THEN salary * 83.5
         WHEN 'UK' THEN salary * 0.79
         WHEN 'Japan' THEN salary * 157.0
         ELSE salary
       END AS local_salary,
       CASE country
         WHEN 'India' THEN 'INR'
         WHEN 'UK' THEN 'GBP'
         WHEN 'Japan' THEN 'JPY'
         ELSE 'USD'
       END AS currency
FROM employees;
```

Q225. GREATEST and LEAST

Easy

Find the highest and lowest among three score columns.

```
SELECT student_name,
       math_score, science_score, english_score,
       GREATEST(math_score, science_score, english_score) AS best,
       LEAST(math_score, science_score, english_score) AS worst
FROM students;
```

Window Functions

Perform calculations across rows without collapsing groups — powerful analytics tool.

Q226. ROW_NUMBER

Easy

Assign a row number to each employee sorted by salary.

```
SELECT ROW_NUMBER() OVER(ORDER BY salary DESC) AS rn,  
       name, salary  
FROM employees;
```

Q227. RANK

Easy

Rank employees by salary (ties get same rank, gaps after).

```
SELECT name, salary,  
       RANK() OVER(ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

Q228. DENSE_RANK

Easy

Dense rank (no gaps) employees by salary.

```
SELECT name, salary,  
       DENSE_RANK() OVER(ORDER BY salary DESC) AS dense_rnk  
FROM employees;
```

Q229. ROW_NUMBER vs RANK vs DENSE_RANK

Medium

Compare all three ranking functions side by side.

```
SELECT name, salary,  
       ROW_NUMBER() OVER(ORDER BY salary DESC) AS row_num,  
       RANK() OVER(ORDER BY salary DESC) AS rank,  
       DENSE_RANK() OVER(ORDER BY salary DESC) AS dense_rank  
FROM employees;
```

Window Functions

Q230. PARTITION BY

Medium

Rank employees within each department by salary.

```
SELECT department, name, salary,  
       RANK() OVER(  
           PARTITION BY department ORDER BY salary DESC  
       ) AS dept_rank  
FROM employees;
```

Q231. LAG

Medium

Show each month's sales alongside the previous month's sales.

```
SELECT month, revenue,  
       LAG(revenue) OVER(ORDER BY month) AS prev_month,  
       revenue - LAG(revenue) OVER(ORDER BY month) AS change  
FROM monthly_sales;
```

Q232. LEAD

Medium

Show each employee's salary and the next higher salary.

```
SELECT name, salary,  
       LEAD(salary) OVER(ORDER BY salary) AS next_salary,  
       LEAD(salary) OVER(ORDER BY salary) - salary AS gap  
FROM employees;
```

Window Functions

Q233. FIRST_VALUE and LAST_VALUE

Medium

Show the highest and lowest salary in each department alongside each row.

```
SELECT department, name, salary,
       FIRST_VALUE(salary) OVER(
           PARTITION BY department ORDER BY salary DESC
       ) AS highest_in_dept,
       LAST_VALUE(salary) OVER(
           PARTITION BY department ORDER BY salary DESC
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS lowest_in_dept
FROM employees;
```

Q234. NTH_VALUE

Medium

Find the 2nd highest salary in each department.

```
SELECT DISTINCT department,
       NTH_VALUE(salary, 2) OVER(
           PARTITION BY department ORDER BY salary DESC
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS second_highest
FROM employees;
```

Q235. NTILE

Medium

Divide employees into 4 equal salary quartiles.

```
SELECT name, salary,
       NTILE(4) OVER(ORDER BY salary) AS quartile
FROM employees;
```

Window Functions

Q236. Running Sum

Medium

Calculate cumulative revenue over time.

```
SELECT order_date, amount,  
       SUM(amount) OVER(ORDER BY order_date) AS running_total  
FROM orders;
```

Q237. Running Average

Medium

Calculate running average salary.

```
SELECT name, salary,  
       ROUND(AVG(salary) OVER(ORDER BY employee_id), 2) AS running_avg  
FROM employees;
```

Q238. Moving Window — Last 3

Medium

3-period moving average of sales.

```
SELECT sale_date, amount,  
       ROUND(AVG(amount) OVER(  
         ORDER BY sale_date  
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
       ), 2) AS moving_avg  
FROM daily_sales;
```

Q239. Percent of Partition Total

Medium

Show each employee's salary as a % of their department total.

```
SELECT department, name, salary,  
       ROUND(salary * 100.0 /  
         SUM(salary) OVER(PARTITION BY department), 2  
       ) AS pct_of_dept  
FROM employees;
```

Window Functions

Q240. Difference from Group Average

Medium

How much each employee's salary differs from their dept average.

```
SELECT department, name, salary,  
       ROUND(AVG(salary) OVER(PARTITION BY department), 2) AS dept_avg,  
       salary - ROUND(AVG(salary) OVER(PARTITION BY department), 2) AS  
diff  
FROM employees;
```

Q241. PERCENT_RANK

Medium

Find each employee's percentile in salary distribution.

```
SELECT name, salary,  
       ROUND(PERCENT_RANK() OVER(ORDER BY salary) * 100, 1) AS  
percentile  
FROM employees;
```

Q242. CUME_DIST

Medium

What percentage of employees earn less than or equal to each salary.

```
SELECT name, salary,  
       ROUND(CUME_DIST() OVER(ORDER BY salary) * 100, 1) AS  
cumulative_pct  
FROM employees;
```

Window Functions

Q243. Named Window

Medium

Reuse window definition across multiple functions.

```
SELECT name, department, salary,  
       RANK() OVER w AS rnk,  
       SUM(salary) OVER w AS running_total,  
       AVG(salary) OVER w AS running_avg  
FROM employees  
WINDOW w AS (PARTITION BY department ORDER BY salary DESC);
```

Q244. Gaps and Islands — Identify Groups

Hard

Find consecutive sequences of active status.

```
WITH numbered AS (  
  SELECT *,  
         ROW_NUMBER() OVER(ORDER BY log_date) -  
         ROW_NUMBER() OVER(PARTITION BY status ORDER BY log_date) AS grp  
  FROM status_log  
)  
SELECT status, MIN(log_date) AS start_date,  
       MAX(log_date) AS end_date,  
       COUNT(*) AS days  
FROM numbered  
GROUP BY status, grp  
ORDER BY start_date;
```

Window Functions

Q245. Top N Per Group with Window

Hard

Get top 3 products by sales in each category.

```
SELECT category, product_name, total_sales
FROM (
    SELECT p.category, p.product_name,
           SUM(oi.quantity * oi.price) AS total_sales,
           ROW_NUMBER() OVER(
               PARTITION BY p.category
               ORDER BY SUM(oi.quantity * oi.price) DESC
           ) AS rn
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.category, p.product_name
) ranked
WHERE rn ≤ 3;
```

Window Functions

Q246. Session Analysis

Hard

Identify sessions: a new session starts after 30+ min gap.

```
WITH gaps AS (  
    SELECT user_id, event_time,  
           EXTRACT(EPOCH FROM event_time -  
                 LAG(event_time) OVER(PARTITION BY user_id ORDER BY  
event_time)  
           ) / 60 AS gap_minutes  
    FROM events  
) ,  
sessions AS (  
    SELECT *,  
           SUM(CASE WHEN gap_minutes > 30 OR gap_minutes IS NULL THEN 1  
ELSE 0 END)  
           OVER(PARTITION BY user_id ORDER BY event_time) AS  
session_id  
    FROM gaps  
)  
SELECT user_id, session_id,  
       MIN(event_time) AS session_start,  
       MAX(event_time) AS session_end,  
       COUNT(*) AS events  
FROM sessions  
GROUP BY user_id, session_id;
```

Window Functions

Q247. Deduplication with ROW_NUMBER

Medium

Keep only the latest record per customer (by updated_at).

```
SELECT * FROM (  
    SELECT *,  
        ROW_NUMBER() OVER(  
            PARTITION BY customer_id  
            ORDER BY updated_at DESC  
        ) AS rn  
    FROM customer_records  
) deduped  
WHERE rn = 1;
```

Q248. Cumulative Count

Medium

Show cumulative new customer count by signup date.

```
SELECT signup_date,  
    COUNT(*) AS new_customers,  
    SUM(COUNT(*)) OVER(ORDER BY signup_date) AS cumulative_total  
FROM customers  
GROUP BY signup_date  
ORDER BY signup_date;
```

Window Functions

Q249. RANGE vs ROWS Frame

Hard

Compare ROWS and RANGE window frames.

```
SELECT order_date, amount,  
       SUM(amount) OVER(  
         ORDER BY order_date  
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
       ) AS rows_sum,  
       SUM(amount) OVER(  
         ORDER BY order_date  
         RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW  
       ) AS range_sum  
FROM orders;
```

Q250. Churn Analysis

Hard

Find customers who haven't ordered in 90+ days (churned).

```
WITH last_orders AS (  
  SELECT customer_id,  
         MAX(order_date) AS last_order_date,  
         CURRENT_DATE - MAX(order_date) AS days_since_last  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT c.customer_name, lo.last_order_date, lo.days_since_last,  
       CASE  
         WHEN lo.days_since_last > 90 THEN 'Churned'  
         WHEN lo.days_since_last > 30 THEN 'At Risk'  
         ELSE 'Active'  
       END AS status  
FROM last_orders lo  
JOIN customers c ON lo.customer_id = c.customer_id  
ORDER BY lo.days_since_last DESC;
```

INSERT, UPDATE, DELETE

Modify data in tables — the write side of SQL.

Q251. INSERT Single Row

Easy

Add a new employee to the table.

```
INSERT INTO employees (name, department, salary, join_date)
VALUES ('Rahul Sharma', 'IT', 55000, '2024-01-15');
```

Q252. INSERT Multiple Rows

Easy

Add 3 employees at once.

```
INSERT INTO employees (name, department, salary) VALUES
('Priya Patel', 'Sales', 45000),
('Amit Kumar', 'IT', 62000),
('Sneha Reddy', 'HR', 48000);
```

Q253. INSERT from SELECT

Medium

Copy all IT employees into an archive table.

```
INSERT INTO employees_archive (name, department, salary)
SELECT name, department, salary
FROM employees
WHERE department = 'IT';
```

Q254. INSERT with RETURNING

Easy

Insert and immediately get the new employee's ID.

```
-- PostgreSQL
INSERT INTO employees (name, salary)
VALUES ('New Employee', 50000)
RETURNING employee_id, name;
```

INSERT, UPDATE, DELETE

Q255. UPDATE Single Column

Easy

Give employee #101 a salary of 65000.

```
UPDATE employees
SET salary = 65000
WHERE employee_id = 101;
```

Q256. UPDATE Multiple Columns

Easy

Change department and salary for an employee.

```
UPDATE employees
SET department = 'Sales',
    salary = 58000
WHERE employee_id = 101;
```

Q257. UPDATE with Calculation

Easy

Give all employees a 10% raise.

```
UPDATE employees
SET salary = salary * 1.10;
```

Q258. UPDATE with Subquery

Medium

Set each employee's bonus to 5% of their department's avg salary.

```
UPDATE employees e
SET bonus = (
    SELECT ROUND(AVG(salary) * 0.05, 2)
    FROM employees e2
    WHERE e2.department = e.department
);
```

INSERT, UPDATE, DELETE

Q259. UPDATE with JOIN

Medium

Update employee city from department table.

```
-- PostgreSQL
UPDATE employees e
SET city = d.city
FROM departments d
WHERE e.department_id = d.department_id;
```

Q260. UPSERT (INSERT ON CONFLICT)

Medium

Insert or update if email already exists.

```
-- PostgreSQL
INSERT INTO employees (email, name, salary)
VALUES ('rahul@company.com', 'Rahul Sharma', 60000)
ON CONFLICT (email)
DO UPDATE SET salary = EXCLUDED.salary,
               name = EXCLUDED.name;
```

Q261. DELETE Specific Rows

Easy

Delete all employees in the 'Temp' department.

```
DELETE FROM employees
WHERE department = 'Temp';
```

INSERT, UPDATE, DELETE

Q262. DELETE with Subquery

Medium

Delete orders older than 2 years for inactive customers.

```
DELETE FROM orders
WHERE order_date < CURRENT_DATE - INTERVAL '2 years'
  AND customer_id IN (
    SELECT customer_id FROM customers
    WHERE is_active = false
  );
```

Q263. DELETE with RETURNING

Medium

Delete and see what was removed.

```
-- PostgreSQL
DELETE FROM employees
WHERE salary < 20000
RETURNING employee_id, name, salary;
```

Q264. TRUNCATE

Easy

Remove all rows from a table quickly.

```
TRUNCATE TABLE temp_data;

-- TRUNCATE is faster than DELETE FROM table
-- It cannot be rolled back in most databases
-- It resets auto-increment counters
```

INSERT, UPDATE, DELETE

Q265. MERGE / UPSERT Pattern

Hard

Sync data from staging table into main table.

```
-- PostgreSQL (INSERT ON CONFLICT)
INSERT INTO products (product_id, name, price, stock)
SELECT product_id, name, price, stock
FROM staging_products
ON CONFLICT (product_id)
DO UPDATE SET
    name = EXCLUDED.name,
    price = EXCLUDED.price,
    stock = EXCLUDED.stock;
```

Table Design & DDL

Create, modify, and manage table structures.

Q266. CREATE TABLE

Easy

Create an employees table with proper data types.

```
CREATE TABLE employees (  
    employee_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    department VARCHAR(50),  
    salary NUMERIC(10,2) DEFAULT 0,  
    join_date DATE DEFAULT CURRENT_DATE,  
    is_active BOOLEAN DEFAULT true,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Q267. CREATE TABLE with Foreign Key

Easy

Create orders table linked to customers.

```
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT NOT NULL,  
    order_date DATE DEFAULT CURRENT_DATE,  
    amount NUMERIC(12,2) NOT NULL,  
    status VARCHAR(20) DEFAULT 'Pending',  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

Table Design & DDL

Q268. ADD Column

Easy

Add a phone column to employees table.

```
ALTER TABLE employees
ADD COLUMN phone VARCHAR(20);
```

Q269. DROP Column

Easy

Remove the fax column from employees table.

```
ALTER TABLE employees
DROP COLUMN fax;
```

Q270. RENAME Column

Easy

Rename 'emp_name' to 'name'.

```
-- PostgreSQL
ALTER TABLE employees
RENAME COLUMN emp_name TO name;
```

Q271. ALTER Column Type

Medium

Change salary from INT to NUMERIC(12,2).

```
ALTER TABLE employees
ALTER COLUMN salary TYPE NUMERIC(12,2);
```

Table Design & DDL

Q272. ADD Constraint

Medium

Add a CHECK constraint: salary must be positive.

```
ALTER TABLE employees
ADD CONSTRAINT chk_salary_positive CHECK (salary > 0);
```

Q273. ADD UNIQUE Constraint

Easy

Ensure no two employees have the same email.

```
ALTER TABLE employees
ADD CONSTRAINT uq_employee_email UNIQUE (email);
```

Q274. CREATE TABLE AS

Easy

Create a backup table from existing data.

```
CREATE TABLE employees_backup AS
SELECT * FROM employees
WHERE is_active = true;
```

Q275. Temporary Table

Easy

Create a temp table for intermediate calculations.

```
CREATE TEMPORARY TABLE temp_salaries AS
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;

SELECT * FROM temp_salaries;
-- Automatically dropped when session ends
```

Table Design & DDL

Q276. Composite Primary Key

Medium

Create enrollment table with composite key.

```
CREATE TABLE enrollments (  
    student_id INT NOT NULL,  
    course_id INT NOT NULL,  
    enrolled_date DATE DEFAULT CURRENT_DATE,  
    grade CHAR(2),  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES students(student_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id)  
);
```

Q277. CREATE SEQUENCE

Medium

Create and use a custom sequence.

```
CREATE SEQUENCE order_seq START 1000 INCREMENT 1;  
  
INSERT INTO orders (order_id, customer_id, amount)  
VALUES (NEXTVAL('order_seq'), 1, 500);  
  
SELECT CURRVAL('order_seq');
```

Q278. DROP TABLE

Easy

Safely drop a table if it exists.

```
DROP TABLE IF EXISTS temp_data;  
  
-- Drop with CASCADE to remove dependent objects  
DROP TABLE IF EXISTS departments CASCADE;
```

Table Design & DDL

Q279. Partition Table

Hard

Create a partitioned table by date range.

```
-- PostgreSQL
CREATE TABLE orders (
    order_id SERIAL,
    order_date DATE NOT NULL,
    amount NUMERIC(12,2)
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2024_q1 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
CREATE TABLE orders_2024_q2 PARTITION OF orders
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Q280. ENUM Type

Medium

Create and use an ENUM type for status.

```
-- PostgreSQL
CREATE TYPE order_status AS ENUM (
    'pending', 'processing', 'shipped', 'delivered', 'cancelled'
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    status order_status DEFAULT 'pending'
);
```

Views, Indexes & Optimization

Performance tuning and reusable query definitions.

Q281. CREATE VIEW

Easy

Create a view showing active employees with department names.

```
CREATE VIEW active_employees AS
SELECT e.employee_id, e.name, d.department_name, e.salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.is_active = true;

SELECT * FROM active_employees;
```

Q282. Materialized View

Medium

Create a materialized view for slow aggregate query.

```
-- PostgreSQL
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS revenue,
       COUNT(*) AS orders
FROM orders
GROUP BY DATE_TRUNC('month', order_date);

-- Refresh when data changes
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue;
```

Views, Indexes & Optimization

Q283. CREATE INDEX

Easy

Speed up queries on frequently searched columns.

```
-- Single column index
CREATE INDEX idx_emp_department ON employees(department);

-- Composite index
CREATE INDEX idx_emp_dept_salary ON employees(department, salary);

-- Unique index
CREATE UNIQUE INDEX idx_emp_email ON employees(email);
```

Q284. Partial Index

Medium

Index only active employees for faster lookups.

```
-- PostgreSQL
CREATE INDEX idx_active_emp ON employees(department, salary)
WHERE is_active = true;
```

Q285. EXPLAIN ANALYZE

Medium

Analyze a query's execution plan.

```
EXPLAIN ANALYZE
SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.salary > 50000;
```

Views, Indexes & Optimization

Q286. Expression Index

Medium

Create index on lowercased email for case-insensitive searches.

```
CREATE INDEX idx_emp_email_lower ON employees(LOWER(email));

-- Now this query uses the index:
SELECT * FROM employees WHERE LOWER(email) = 'john@company.com';
```

Q287. GIN Index for Full-Text Search

Hard

Set up full-text search on product descriptions.

```
-- PostgreSQL
ALTER TABLE products ADD COLUMN search_vector tsvector;

UPDATE products
SET search_vector = to_tsvector('english', name || ' ' || description);

CREATE INDEX idx_product_search ON products USING GIN(search_vector);

SELECT product_name, description
FROM products
WHERE search_vector @@ to_tsquery('english', 'wireless & bluetooth');
```

Views, Indexes & Optimization

Q288. Query Optimization Tips

Medium

Rewrite a slow query for better performance.

```
-- SLOW: Using function on indexed column
SELECT * FROM orders WHERE YEAR(order_date) = 2024;

-- FAST: Use range comparison instead
SELECT * FROM orders
WHERE order_date ≥ '2024-01-01' AND order_date < '2025-01-01';

-- SLOW: SELECT * when you need few columns
SELECT * FROM employees WHERE department = 'IT';

-- FAST: Select only needed columns
SELECT name, salary FROM employees WHERE department = 'IT';

-- SLOW: NOT IN with NULLs
SELECT * FROM a WHERE id NOT IN (SELECT id FROM b);

-- FAST: NOT EXISTS
SELECT * FROM a WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.id = a.id);
```

Q289. Vacuum and Analyze

Medium

Maintain table statistics for the optimizer.

```
-- PostgreSQL: Reclaim space and update statistics
VACUUM ANALYZE employees;

-- Check table bloat
SELECT schemaname, tablename,

pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
total_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;
```

Views, Indexes & Optimization

Q290. Transaction Control

Medium

Use transactions for safe multi-statement operations.

```
BEGIN;

UPDATE accounts SET balance = balance - 5000
WHERE account_id = 101;

UPDATE accounts SET balance = balance + 5000
WHERE account_id = 202;

-- If both succeed:
COMMIT;

-- If something goes wrong:
-- ROLLBACK;

-- Savepoint example:
BEGIN;
SAVEPOINT sp1;
UPDATE employees SET salary = 0 WHERE employee_id = 1;
ROLLBACK TO SAVEPOINT sp1;
COMMIT;
```

Advanced & Real-World Problems

Complex real-world scenarios combining multiple SQL concepts.

Q291. Customer Lifetime Value

Hard

Calculate customer lifetime value with recency, frequency, monetary.

```
WITH rfm AS (  
    SELECT customer_id,  
           CURRENT_DATE - MAX(order_date) AS recency_days,  
           COUNT(*) AS frequency,  
           SUM(amount) AS monetary  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT c.customer_name, r.recency_days, r.frequency, r.monetary,  
       NTILE(5) OVER(ORDER BY r.recency_days DESC) AS r_score,  
       NTILE(5) OVER(ORDER BY r.frequency) AS f_score,  
       NTILE(5) OVER(ORDER BY r.monetary) AS m_score  
FROM rfm r  
JOIN customers c ON r.customer_id = c.customer_id  
ORDER BY r.monetary DESC;
```

Advanced & Real-World Problems

Q292. Funnel Analysis

Hard

Analyze user funnel: visit → signup → purchase → repeat.

```
WITH funnel AS (  
    SELECT  
        COUNT(DISTINCT CASE WHEN event = 'visit' THEN user_id END) AS  
visits,  
        COUNT(DISTINCT CASE WHEN event = 'signup' THEN user_id END) AS  
signups,  
        COUNT(DISTINCT CASE WHEN event = 'purchase' THEN user_id END) AS  
purchases,  
        COUNT(DISTINCT CASE WHEN event = 'repeat_purchase' THEN user_id  
END) AS repeat_buys  
    FROM events  
    WHERE event_date ≥ CURRENT_DATE - INTERVAL '30 days'  
)  
SELECT visits, signups, purchases, repeat_buys,  
    ROUND(signups * 100.0 / NULLIF(visits, 0), 1) AS visit_to_signup,  
    ROUND(purchases * 100.0 / NULLIF(signups, 0), 1) AS  
signup_to_purchase,  
    ROUND(repeat_buys * 100.0 / NULLIF(purchases, 0), 1) AS  
purchase_to_repeat  
FROM funnel;
```

Advanced & Real-World Problems

Q293. Inventory Reorder Alert

Hard

Find products that need reordering based on daily sales rate.

```
WITH daily_sales AS (  
    SELECT product_id,  
           AVG(quantity) AS avg_daily_sales,  
           SUM(quantity) AS total_sold  
    FROM order_items  
    WHERE order_date ≥ CURRENT_DATE - INTERVAL '30 days'  
    GROUP BY product_id  
)  
SELECT p.product_name, p.stock_quantity,  
       ROUND(ds.avg_daily_sales, 1) AS avg_daily_sales,  
       ROUND(p.stock_quantity / NULLIF(ds.avg_daily_sales, 0)) AS  
days_of_stock,  
       CASE  
           WHEN p.stock_quantity / NULLIF(ds.avg_daily_sales, 0) < 7  
THEN 'URGENT'  
           WHEN p.stock_quantity / NULLIF(ds.avg_daily_sales, 0) < 14  
THEN 'REORDER'  
           ELSE 'OK'  
       END AS alert  
FROM products p  
JOIN daily_sales ds ON p.product_id = ds.product_id  
ORDER BY days_of_stock ASC;
```

Advanced & Real-World Problems

Q294. Employee Attrition Analysis

Hard

Monthly attrition rate by department.

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', leave_date) AS month,  
           department,  
           COUNT(*) AS left_count  
    FROM employees  
    WHERE leave_date IS NOT NULL  
    GROUP BY DATE_TRUNC('month', leave_date), department  
) ,  
headcount AS (  
    SELECT DATE_TRUNC('month', d) AS month, e.department,  
           COUNT(*) AS total  
    FROM GENERATE_SERIES('2023-01-01', '2024-12-31', '1 month') d  
    CROSS JOIN LATERAL (  
        SELECT DISTINCT department FROM employees  
    ) e(department)  
    LEFT JOIN employees emp  
        ON emp.department = e.department  
        AND emp.join_date ≤ d  
        AND (emp.leave_date IS NULL OR emp.leave_date > d)  
    GROUP BY DATE_TRUNC('month', d), e.department  
)  
SELECT h.month, h.department, h.total,  
       COALESCE(m.left_count, 0) AS left,  
       ROUND(COALESCE(m.left_count, 0) * 100.0 / NULLIF(h.total, 0), 2)  
       AS attrition_pct  
FROM headcount h  
LEFT JOIN monthly m ON h.month = m.month AND h.department = m.department  
ORDER BY h.month, h.department;
```

Advanced & Real-World Problems

Q295. Market Basket Analysis

Hard

Find products frequently bought together.

```
SELECT p1.product_name AS product_a,  
       p2.product_name AS product_b,  
       COUNT(*) AS times_bought_together  
FROM order_items oi1  
JOIN order_items oi2  
      ON oi1.order_id = oi2.order_id  
      AND oi1.product_id < oi2.product_id  
JOIN products p1 ON oi1.product_id = p1.product_id  
JOIN products p2 ON oi2.product_id = p2.product_id  
GROUP BY p1.product_name, p2.product_name  
HAVING COUNT(*) ≥ 5  
ORDER BY times_bought_together DESC  
LIMIT 20;
```

Advanced & Real-World Problems

Q296. Cohort Retention Analysis

Hard

Track monthly retention by signup cohort.

```
WITH cohort AS (  
    SELECT customer_id,  
           DATE_TRUNC('month', MIN(order_date)) AS cohort_month  
    FROM orders GROUP BY customer_id  
)  
activity AS (  
    SELECT o.customer_id, c.cohort_month,  
           DATE_TRUNC('month', o.order_date) AS activity_month,  
           EXTRACT(MONTH FROM AGE(  
               DATE_TRUNC('month', o.order_date), c.cohort_month  
           )) AS month_number  
    FROM orders o  
    JOIN cohort c ON o.customer_id = c.customer_id  
)  
SELECT cohort_month,  
       month_number,  
       COUNT(DISTINCT customer_id) AS active_users,  
       ROUND(COUNT(DISTINCT customer_id) * 100.0 /  
             FIRST_VALUE(COUNT(DISTINCT customer_id))  
              OVER(PARTITION BY cohort_month ORDER BY month_number), 1  
       ) AS retention_pct  
FROM activity  
GROUP BY cohort_month, month_number  
ORDER BY cohort_month, month_number;
```

Advanced & Real-World Problems

Q297. Slow Query Identification

Hard

Find the slowest queries in PostgreSQL.

```
-- PostgreSQL (requires pg_stat_statements extension)
SELECT query,
       calls,
       ROUND(total_exec_time::NUMERIC, 2) AS total_ms,
       ROUND(mean_exec_time::NUMERIC, 2) AS avg_ms,
       rows
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Q298. Dynamic Pivot Table

Hard

Dynamically pivot sales data by product (using CROSSTAB).

```
-- PostgreSQL (requires tablefunc extension)
CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT * FROM CROSSTAB(
    'SELECT month, product_name, total_sales
     FROM monthly_product_sales
     ORDER BY 1, 2',
    'SELECT DISTINCT product_name
     FROM monthly_product_sales ORDER BY 1'
) AS ct(
    month DATE,
    "Product A" NUMERIC,
    "Product B" NUMERIC,
    "Product C" NUMERIC
);
```

Advanced & Real-World Problems

Q299. JSON Operations

Hard

Work with JSON data stored in PostgreSQL.

```
-- Create table with JSON column
CREATE TABLE events (
    id SERIAL PRIMARY KEY,
    data JSONB NOT NULL
);

-- Insert JSON data
INSERT INTO events (data) VALUES
('{"user": "john", "action": "login", "device": "mobile"}');

-- Query JSON fields
SELECT data->>'user' AS username,
       data->>'action' AS action,
       data->'device' AS device_json
FROM events
WHERE data->>'action' = 'login';

-- JSON aggregation
SELECT data->>'action' AS action,
       COUNT(*) AS count
FROM events
GROUP BY data->>'action';

-- Search within JSON arrays
SELECT * FROM events
WHERE data @> '{"tags": ["urgent"]}';
```

Build a complete business dashboard in one query with CTEs.

```
WITH
revenue AS (
    SELECT SUM(amount) AS total_revenue,
           SUM(CASE WHEN order_date ≥ DATE_TRUNC('month', CURRENT_DATE)
                    THEN amount ELSE 0 END) AS mtd_revenue,
           COUNT(*) AS total_orders,
           COUNT(DISTINCT customer_id) AS unique_customers
    FROM orders
    WHERE order_date ≥ DATE_TRUNC('year', CURRENT_DATE)
),
top_products AS (
    SELECT p.product_name,
           SUM(oi.quantity * oi.price) AS revenue,
           ROW_NUMBER() OVER(ORDER BY SUM(oi.quantity * oi.price) DESC)
    AS rn
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.product_name
),
growth AS (
    SELECT
        SUM(CASE WHEN order_date ≥ DATE_TRUNC('month', CURRENT_DATE)
                 THEN amount END) AS this_month,
        SUM(CASE WHEN order_date ≥ DATE_TRUNC('month', CURRENT_DATE)
                 - INTERVAL '1 month'
                 AND order_date < DATE_TRUNC('month', CURRENT_DATE)
                 THEN amount END) AS last_month
    FROM orders
)
SELECT
    r.total_revenue, r.mtd_revenue,
    r.total_orders, r.unique_customers,
    ROUND(r.total_revenue / NULLIF(r.unique_customers, 0), 2) AS
avg_per_customer,
    ROUND((g.this_month - g.last_month) * 100.0
          / NULLIF(g.last_month, 0), 1) AS mom_growth_pct,
    (SELECT STRING_AGG(product_name, ', ')
     FROM top_products WHERE rn ≤ 5) AS top_5_products
FROM revenue r, growth g;
```